MyGES

Application Mobile Campus Eductive Aix-en-Provence

Jetpack Compose • Kotlin • Clean Architecture

Contexte du Projet

Objectif

Centraliser les services essentiels du campus pour les étudiants d'Aix-en-Provence

Services Implémentés

- 📰 Actualités • 📅 Emploi du temps
- 📊 Notes & Examens • 👤 Profil

✅ Exigences Obligatoires

- UI/UX Jetpack Compose
- Intégration API MyGES
- Internationalisation (EN/FR)

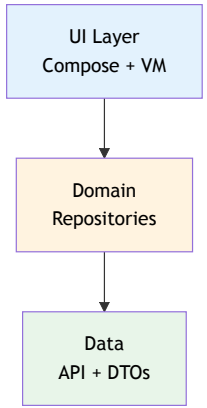
Exigences Optionnelles

- ✅ Authentification OAuth2
- ❌ Base locale (Room)
- ❌ Firebase / Cartographie

Architecture Technique

Clean Architecture + MVVM

3 Couches



Stack Technologique

UI & Core

- Compose 2025.11 • Material 3
- Navigation Compose

Réseau

- Retrofit 3.0 • OkHttp 5.3
- Moshi 1.15 • Coil 3.0

Outils

- Koin 4.1 (DI)
- Android Keystore (Sécurité)

Structure du Projet

Arborescence Clean Architecture

```
app/src/main/java/com/myges/
├── core/di/          # Koin
├── data/             # Data Layer
│   ├── dtos/
│   ├── mappers/
│   ├── network/
│   └── repositories/
├── domain/           # Domain Layer
│   ├── entities/
│   └── repositories/
└── ui/               # UI Layer
    ├── common/
    ├── theme/
    └── screens/
```

Organisation par Feature

```
screens/home/
├── HomeScreen.kt
└── viewModel/
    └── HomeViewModel.kt
```

HomeScreen.kt contient :

- `HomeScreen()` - Smart
- `HomeContent()` - Dumb
- `LoadingContent()`, `ErrorContent()`
- NewsCard, BannersPager…

✅ Tout dans un fichier
✅ Facile à naviguer

Pattern MVVM - Exemple de Code

HomeScreen (View)

```
@Composable
fun HomeScreen(
  viewModel: HomeViewModel
) {
  val uiState by viewModel
    .uiState.collectAsState()

  when (uiState) {
    Loading → LoadingScreen()
    is Success → NewsContent( ... )
    is Error → ErrorScreen( ... )
  }
}
```

HomeViewModel

```
class HomeViewModel(
  private val repository:
    NewsRepository
) : ViewModel() {

  private val _uiState =
    MutableStateFlow<UiState>(Loading)
  val uiState: StateFlow<UiState>

  init {
    viewModelScope.launch {
      val news = repository.getNews()
      _uiState.value = Success(news)
    }
  }
}
```

Gestion d'État Type-Safe

```
sealed interface HomeUiState {
    data object Loading : HomeUiState

    data class Success(
        val news: List<News>,
        val banners: List<Banner>
    ) : HomeUiState

    data class Error(val message: String) : HomeUiState
}
```

✅ **Type-safe** • ✅ **Exhaustif** • ✅ **Maintenable**

Sécurité : OAuth2 + Android Keystore (AES-GCM 256 bits) pour chiffrement des tokens

Navigation Type-Safe

Bootstrap Navigation

```
@Composable
fun BootstrapNavigation() {
  val state by userRepository
    .appState.collectAsState()

  Crossfade(state) { appState →
    when (appState) {
      SPLASH → SplashScreen()
      UNAUTHENTICATED → SignInScreen()
      AUTHENTICATED → AuthenticatedNav()
    }
  }
}
```

Flow Global : Splash → Connexion → App

Routes Sérialisables

```
@Serializable
object HomeRoute

@Serializable
data class ProfileRoute(
  val userId: String? = null
)
```

Navigation

```
navController.navigate(
  ProfileRoute(userId = "123")
)
```

✅ **Type-safe** • ✅ **Validation auto** • ✅ **Pas de strings hardcodées**

Gestion API & DTOs

Architecture Réseau

Retrofit + Moshi + OkHttp

```
interface ApiService {
  @GET("/me/news")
  suspend fun getNews(): List<NewsDto>

  @GET("/me/profile")
  suspend fun getProfile(): UserDto
}
```

Repository

```
class NewsRepositoryImpl(
  private val api: ApiService
) : NewsRepository {
  override suspend fun getNews() =
    api.getNews().map { it.toDomain() }
}
```

Mapping DTO → Domain

```
@JsonClass(generateAdapter = true)
data class UserDto(
  val id: String,
  val name: String,
  val email: String
)
```

Mapper

```
fun UserDto.toDomain() = User(
  id = UUID.fromString(id),
  name = name,
  email = email
)
```

✅ **Séparation API/Métier**
✅ **Validation des types**
✅ **Pas de logique dans les DTOs**

Fonctionnalités - 4 Écrans Principaux

🏠 Actualités

- Carousel bannières • Flux actualités
- Pull-to-refresh • Cache images

`GET /me/news` • `GET /me/news/banners`

📅 Emploi du Temps

- Vue par date • Détails complets
- Badge type (TD/CM/TP) • Prof, salle

`GET /me/agenda?start={timestamp}`

📊 Notes & Examens

- Par année académique
- Filtrage trimestre (T1/T2/T3)
- Note + moyenne classe

`GET /me/{year}/grades`

👤 Profil Étudiant

- Infos personnelles & académiques
- Programme & promotion
- Contact d'urgence

`GET /me/profile`

Interface & Design

Material Design 3 + Composants

Material Design 3

✅ **Dynamic Color** (Android 12+)
Couleurs adaptées au fond d'écran

✅ **Dark Mode**
Auto-détection système

✅ **Edge-to-Edge**
Plein écran avec gestion insets

Pattern Smart/Dumb

Smart (Screens)

- Gèrent état via ViewModel
- Orchestrent navigation

Dumb (Content)

- Présentation pure
- Réutilisables & testables

Composants: LoadingScreen, ErrorScreen, NewsCard, AgendaItem, GradeCard

Pattern Smart/Dumb Components

Smart Component (Screen)

```
@Composable
fun HomeScreen(
  viewModel: HomeViewModel = koinViewModel()
) {
  val state by viewModel.uiState
    .collectAsStateWithLifecycle()

  HomeContent(
    state = state,
    onRefresh = viewModel::refresh
  )
}
```

- Gestion du ViewModel
- Cycle de vie
- Navigation

Dumb Component (Content)

```
@Composable
fun HomeContent(
  state: HomeUiState,
  onRefresh: () → Unit
) {
  LazyColumn {
    // UI pure sans logique métier
  }
}

@Preview
@Composable
private fun Preview() {
  AppTheme {
    HomeContent(HomeUiState(), {})
  }
}
```

- ✅ Testable • ✅ Prévisualisable • ✅ Réutilisable

Internationalisation (EN/FR)

Support Multilingue

🇬🇧 **Anglais** (défaut) • 🇫🇷 **Français**

```
res/
├── values/
│   └── strings.xml        # EN
└── values-fr/
    └── strings.xml        # FR
```

✅ Tous les textes externalisés
✅ Auto-détection locale système

Exemples strings.xml

```
<!— values/strings.xml (EN) —>
<string name="home_title">Home</string>
<string name="signin_button">Sign In</string>
<string name="loading">Loading…</string>

<!— values-fr/strings.xml (FR) —>
<string name="home_title">Accueil</string>
<string name="signin_button">Se connecter</string>
<string name="loading">Chargement…</string>
```

Utilisation

```
Text(stringResource(R.string.home_title))
```

Démonstration Live

Parcours Utilisateur

Connexion OAuth2 → Actualités → Emploi du Temps → Notes → Profil

Switch EN/FR • Dark Mode • Pull-to-Refresh

Questions ?

4ESGI AIX • Jetpack Compose • Clean Architecture

MyGES - Application Mobile Campus Eductive

Merci pour votre attention !